



BioHackathon series:
[Eurobioc 2026 Hackathon](#)
Turku, Finland, 2023
[Group 1](#)

Submitted: 02 Jun 2026

License:
Authors retain copyright and
release the work under a Creative
Commons Attribution 4.0
International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

GPU Access in R

James Dagleish ¹, Sebastian Fischer ², and Nicholas P Cooley ³

¹ Oxford University ² LMU Munich ³ University of Limerick

Introduction

The S language was originally written by Jonathan Chambers in 1976(<https://journal.r-project.org/articles/RJ-2020-028/RJ-2020-028.pdf>), building upon the multics and BLISS (Bell Labs Interactive Statistical System) project efforts previous efforts involving John Tukey. Thus, from the start, the statistical language S was designed with interactive use in mind (hence the I in the name BLISS), thereby lending itself to smaller problems and exploratory data analysis that precedes statistical testing. R language, derived from S, was first developed by Robert Gentleman and Ross Ihaka (<https://doi.org/10.2307%2F1390807>) and originally mentioned one of the key improvements was “memory management.” GPU computing derives its success in part by careful memory management that allows for rapid communication between threads within a streaming multiprocessor. GPU computing is a key advantage of the most successful scripting languages for general use, though GPU use in R has not seen dramatic uptake. Surprisingly, the idea to use R with GPU capabilities is not new. The first GPU packages were written as far back as 2009, with the gputools package (<https://pubmed.ncbi.nlm.nih.gov/19850754/>) and NVIDIA training materials for R were made in 2014 (<https://developer.nvidia.com/blog/accelerate-r-applications-cuda/>). The reasons for this include the small population size of packages on R, lack of maintenance, the typical R use case, and the CRAN platform’s rapid changes that makes package maintenance challenging over time. The number of packages in the CRAN archive that are gpu based is only 10 out of the total 23797, with most of them defunct and removed from CRAN. [GPUMatrix](#), [h2o4gpu](#), and [GeneralizedUmatrixGPU](#) still function while gputools, gpuR, rpud, nmfgpu4R were archived. One package permGPU functions in an OS-specific manner and another nmfgpu4r still actually builds but has warnings. Given the popularity of hardware acceleration outside of academia, and in non-academic settings, this lack of CRAN provided packages can be viewed as perplexing. Several reasons exist why the adoption has been limited, including the three layer requirement for packages based on conda environments to use popular python GPU-interfaces and the incompatibility of R with older versions of packages. We believe the first could be solved by using CUDA C++ and C external wrapping functions with R (which an initial project has been built, ACF) and the latter could be solved by allowing a system for outdated R dependencies to be incorporated from the CRAN archive.

Motivation

The inaccessibility of hardware acceleration in the R language has been a missing feature that most R users have been content to live without. The rugged landscape of compute frameworks and hardware providers have also made it difficult for the R community to come together on a paradigm for access that would fit well within R as a language and R as a research ecosystem. Bringing hardware acceleration into R, particularly with CUDA and NVIDIA devices would improve the value of R as a skill for graduates leaving academia, increase the throughput and efficiency of research computing in R, and expand the versatility of R generally.



Results

This hackathon project produced a series of observations about the current state of packages in CRAN that provide access to GPU compute. It leveraged this context to also improve documentation and usability the R packages ACFcuda, and cudatoolkit.

Package	Description
ACFcuda	Basic access to nvidia devices through the CUDA and runtime APIs
cudatoolkit	please add a description sebastian!

Meeting information

If you want to submit a preprint to BioHackrXiv, first check if your meeting is registered. You can find a list of meetings [here](#). If your meeting is missing, please contact your meeting organizers. The above list also provides information on the YAML fields with information about the meeting.

The following fields need to be given:

```
biohackathon_name: "BioHackathon Europe 2023"
biohackathon_url:  "https://biohackathon-europe.org/"
biohackathon_location: "Barcelona, Spain, 2023"
group: Project 26
git_url: https://github.com/yourOrganization/your_report_repo
```

The [BioHackrXiv meeting pages](#) provide content to use for the first three fields. The `git_url` field must have the link to the GitHub repository with your preprint (draft).

Author information

Information about the authors is given in the [YAML](#) format at the top of this template. For authors you provide their names, their affiliations. That is the minimum, but as BioHackrXiv is moving to a situation where more metadata is shared, and used by, for example, EuropePMC, adding additional information is encouraged.

BioHackathons is about hacking together, and the minimal number of authors for reports is two. This makes a minimal example look like this:

```
authors:
  - name: First Author
    affiliation: 1
  - name: Last Author
    affiliation: 2
affiliations:
  - name: First Affiliation
    index: 1
  - name: ELIXIR Europe
    index: 2
```

Author identifiers

Ideally, authors provide their [ORCID](#) identifier. For affiliations, It is added with the `orcid` field. So, an author record would look like this:

```
authors:
  - name: First Author
    affiliation: 1
    orcid: 0000-0000-0000-0000
```

Research Organization Registry identifiers

Matching the author identifier, the affiliations can be further specified with the [Research Organization Registry](#) (ROR) identifier. For example, this is the affiliation identifier can be added with the `ror:` field:

```
affiliations:
- name: ELIXIR Europe
  ror: 044rwnt51
  index: 2
```

Contributor Role Taxonomy

A last feature since is minimal support for the Contributor Role Taxonomy (CRediT). You can specify the role of authors in writing the report with the `role:` field. However, the authors are responsible for selection the right terms from [CRediT](#). An example looks like this:

```
authors:
- name: First Author
  affiliation: 1
  orcid: 0000-0000-0000-0000
  role: Conceptualization, Writing - review & editing
```

A full examples

A full example then has this structure:

```
authors:
- name: First Author
  affiliation: 1
  role: Writing - original draft
- name: Last Author
  orcid: 0000-0000-0000-0000
  affiliation: 2
  role: Conceptualization, Writing - review & editing
affiliations:
- name: First Affiliation
  index: 1
- name: ELIXIR Europe
  ror: 044rwnt51
  index: 2
```

Formatting

This document use Markdown and you can look at [this tutorial](#).

Subsection level 2

Please keep sections to a maximum of only two levels.

Tables

Tables can be added in the following way, though alternatives are possible:

Table: Note that table caption is automatically numbered and should be given before the table itself.



Header 1	Header 2
-----	-----
item 1	item 2
item 3	item 4

This gives:

Table 2: Note that table caption is automatically numbered and should be given before the table itself.

Header 1	Header 2
item 1	item 2
item 3	item 4

Figures

A figure is added with:

`![Caption for BioHackrXiv logo figure](./biohackrxiv.png)`

This gives:



Figure 1: Caption for BioHackrXiv logo figure

Figures can be scaled by adding the width or height to the Markdown like this:

`![Caption for BioHackrXiv logo figure](./biohackrxiv.png){ width=50px }`

Other main section on your manuscript level 1

Lists can be added with:

1. Item 1
2. Item 2

Citation Typing Ontology annotation

You can use [CiTO](#) annotations, as explained in [this BioHackathon Europe 2021 write up](#) and [this CiTO Pilot](#). Using this template, you can cite an article and indicate *why* you cite that article, for instance DisGeNET-RDF (Queralt-Rosinach et al., 2016).

The syntax in Markdown is as follows: a single intention annotation looks like `[@usesMethod In:Krewinkel2017]`; two or more intentions are separated with colons, like `[@extends:dis`

cusses:Nielsen2017Scholia]. When you cite two different articles, you use this syntax: [
@citesAsDataSource:Ammar2022ETL; @citesAsDataSource:Arend2022BioHackEU22].

Possible CiTO typing annotation include:

- citesAsDataSource: when you point the reader to a source of data which may explain a claim
- usesDataFrom: when you reuse somehow (and elaborate on) the data in the cited entity
- usesMethodIn
- citesAsAuthority
- citesAsEvidence
- citesAsPotentialSolution
- citesAsRecommendedReading
- citesAsRelated
- citesAsSourceDocument
- citesForInformation
- confirms
- documents
- providesDataFor
- obtainsSupportFrom
- discusses
- extends
- agreesWith
- disagreesWith
- updates
- citation: generic citation

Results

Discussion

...

Acknowledgements

...

References

Queralt-Rosinach, N., Pinero, J., Bravo, À., Sanz, F., & Furlong, L. I. (2016). DisGeNET-RDF: harnessing the innovative power of the Semantic Web to explore the genetic basis of diseases. *Bioinformatics*, 32(14), 2236–2238. <https://doi.org/10.1093/bioinformatics/btw214> [cito:citesAsAuthority]

Author contributions

James Dagleish: Author Sebastian Fischer: Author Nicholas P Cooley: Author